15章

イベント駆動型アーキテクチャ

マイクロサービスと同じように、イベント駆動型アーキテクチャ（Event Driven Architecture）は、現在の分散システムで広く使われています。分散システムを疎結合に保ち、拡張性と耐障害性を向上するための有力な技術方式として、イベント駆動型アーキテクチャを推奨する人がたくさんいます。

イベント駆動型アーキテクチャは、たいていの場合、ドメイン駆動設計と関係します。結局のところ、イベント駆動型アーキテクチャは、名前のとおり、イベントにもとづく技術方式です。そしてドメイン駆動設計でも、イベントは重要な構成要素です。事業活動を業務イベントで表現し、もし必要であれば、イベント履歴を「真実を語る唯一の情報源」として扱います。イベント駆動型アーキテクチャでシステムを構築するために、ドメイン駆動設計の業務イベントを活用したくなるかもしれません。しかし、これはよいアイデアでしょうか？

イベントを秘伝のスパイスとして既存システムに振りかければ、既存システムが疎結合な分散型システムに変身するわけではありません。まったく逆です。つまり、イベント駆動型アーキテクチャを安易に取り入れると、「モジュラーモノリス」（モジュール化された巨大な一枚岩）を「分散した大きな泥団子」に変えてしまうかもしれません。

この章では、イベント駆動型アーキテクチャとドメイン駆動設計の関係を検討します。イベント駆動型アーキテクチャの基本的な構成要素、この方式で失敗する典型的な状況、そして、効果的な非同期式の連係を設計するために、どのようにドメイン駆動設計の手法を活用できるかを学びます。

15.1　イベント駆動型アーキテクチャとは？

簡単に言えば、イベント駆動型アーキテクチャとは、システムのコンポーネント間で、イベントメッセージを非同期でやりとりする技術方式です（図15‐1）。他のコンポーネントのエンドポイントを同期式で呼び出すのではなく、コンポーネントは、業務領域で何か変化が起きたことを他のコンポーネントに通知するために、イベントを発行（publish）します。他のコンポーネントはシステムで起きた変化をイベントとして購読（subscribe）し、そのイベントに応じた動作をします。第9章で説明したサーガは、典型的なイベント駆動型アーキテクチャです。

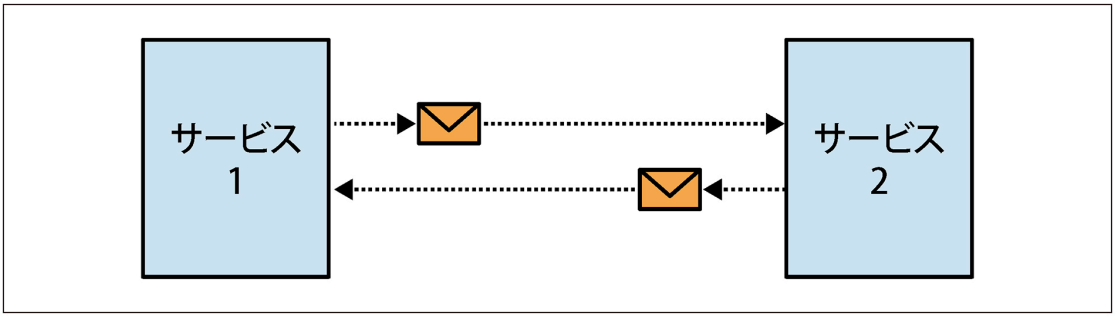

図15-1　非同期通信

イベント駆動型アーキテクチャとイベントソーシングの違いをはっきりさせておきましょう。第7章で検討したように、イベントソーシングは、状態の変化を一連のイベントとして捕捉する方法です。

イベント駆動型アーキテクチャとイベントソーシングは、どちらもイベント（出来事）に注目します。しかし二つの方式は異なる考え方です。イベント駆動型アーキテクチャはサービス間の通信方法です。一方で、イベントソーシングはサービス内部の実装方法です。イベントソーシングのイベントは、サービスの内部状態の変化（イベント履歴式ドメインモデルの集約の状態の変化）を表現します。イベントソーシングは事業活動の複雑さを把握するための方法です。システムの他のコンポーネントと連係することを意図した方法ではありません。

後ほど説明しますが、イベントは三つのカテゴリーに分類できます。サービスどうしを連係するためには、あるカテゴリーのイベントは他のカテゴリーのイベントより適しています。

15.2　イベント

イベント駆動型アーキテクチャを採用したシステムでは、イベントを送ることでコンポーネントどうしをつなぎ、一つのシステムとして動作させます。まずイベントについて詳しく見てみましょう。そしてイベントとメッセージの違いも押さえておきましょう。

15.2.1　イベント、コマンド、メッセージ

ここまで説明した内容だと、イベントはメッセージの定義＊1と似ています。しかしこの二つは同じではありません。イベントはメッセージです。しかし、メッセージの種類はイベントだけではありません。メッセージは、イベントとコマンドの二つに分類できます。

＊1　G. Hohpe、B. Woolf共著『Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions』（Addison-Wesley、2003年）

イベント

すでに起こった変化を表現したメッセージ

コマンド

これから実行すべき操作を表現したメッセージ

イベントは過去に起きた何かです。それに対しコマンドはこれからやるべき何かを示します。イベントとコマンドのどちらも非同期メッセージとして送信されます。しかしコマンドは拒否される可能性があります。つまり、コマンドを受け取った側は、たとえば、内容が不完全なコマンドや業務ルールに違反するコマンドを受け入れない可能性があります。それに対し、イベントを受け取った側はイベントを取り消せません。イベントはすでに起きた事実です。イベントを受け取った側が、イベントに対して取り消しなど逆方向の処理をする唯一の手段は、サーガの補償アクションの起動、つまりコマンドの発行です。

イベントはすでに起きた事実です。ですからイベント名は過去形で表現します。たとえば、`DeliveryScheduled`（配送手配済）、`ShipmentCompleted`（出荷済）、`DeliveryConfirmed`（配送確認済）などです。

15.2.2　データ構造

イベントは、何らかのメッセージ通信基盤を使って、内部の表現形式を通信用の表現形式に直列化（serialized）して外部に送信可能なデータレコードです。イベントのデータ構造は、典型的には、イベントのメタデータ（イベントを説明する付加的な情報）とペイロード（イベントの内容）で構成します。たとえば以下の構造です。

```
{
  "type": "delivery-confirmed",
  "event-id": "14101928-4d79-4da6-9486-dbc4837bc612",
  "correlation-id": "08011958-6066-4815-8dbe-dee6d9e5ebac",
  "delivery-id": "05011927-a328-4860-a106-737b2929db4e",
  "timestamp": 1615718833,
  "payload": {
    "confirmed-by": "17bc9223-bdd6-4382-954d-f1410fd286bd",
    "delivery-time": 1615701406
  }
}
```

イベントのペイロードは、イベントの内容だけでなく、イベントの種類も表現しています。イベントを三つのカテゴリーに分類し、それぞれのカテゴリーのイベントの特徴と違いについて検討してみましょう。

15.2.3　イベントのカテゴリー

イベントは次の三つのカテゴリーに分類できます[＊2]。

[＊2]　M. Fowler, “What do you mean by “Event-Driven”?”, Martin Fowler (blog)(https://oreil.ly/aSK5l)（2021年8月12日参照）

- イベント通知
- イベントによる状態転送
- 業務イベント

15.2.3.1　イベント通知

イベント通知は、他のコンポーネントが反応することを想定した、事業活動で起きた変化に関するメッセージです。たとえば、`PayCheckGenerated`（給

与明細発行済）通知や`CampaignPublished`（キャンペーン公開済）通知です。

イベント通知の内容は必要最小限です。そのイベントに関心のある相手、つまりイベントの購読者にイベントの発生を伝えることが唯一の目的です。イベントに反応するために必要なすべての情報を、イベント通知に含めてはいけません。イベント通知の例は次のとおりです。

```
{
  "type": "paycheck-generated",
  "event-id": "537ec7c2-d1a1-2005-8654-96aee1116b72",
  "delivery-id": "05011927-a328-4860-a106-737b2929db4e",
  "timestamp": 1615726445,
  "payload": {
    "employee-id": "456123",
    "link": "/paychecks/456123/2021/01"
  }
}
```

このイベントは、給与明細を発行したことを外部のコンポーネントに通知します。給与明細に関するすべての情報は送っていません。そのかわり、この通知を受け取った側は、通知に含まれるリンクを使って、詳細な情報を入手します。このやりとりは図15‐2のようになります。

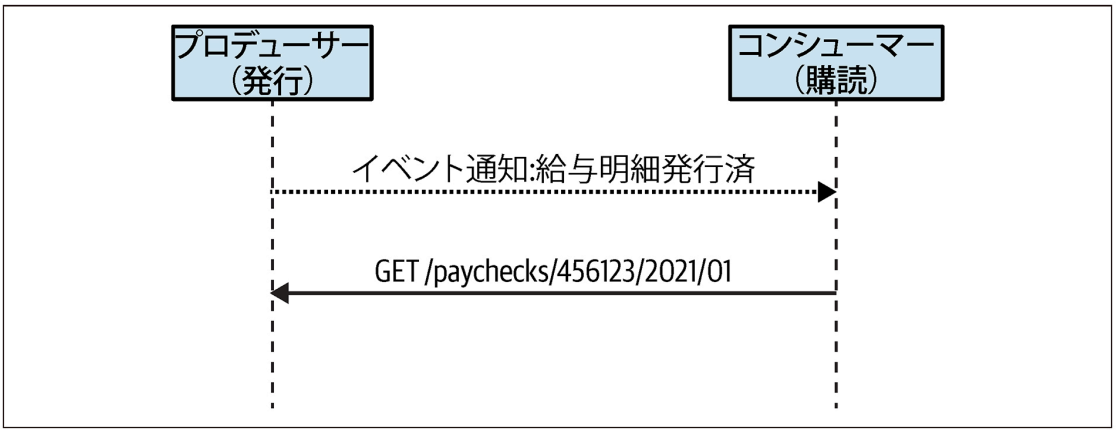

図15-2　イベント通知の流れ

イベント通知を使ったこのやり方は、ある意味で地震や災害の緊急速報メッセージと似ています＊3。携帯電話のショートメッセージを使って、地震や災害を一斉同報します。このシステムで送信できる文字数は数百文字に制限され

ています。この短いメッセージでも緊急事態が発生したことを伝えるには十分です。しかし、詳細な情報は、別の情報源から入手しなければなりません。

＊3　［訳注］原文はアメリカ合衆国のWEA：Wireless Emergency Alert、ヨーロッパのEU-Alert。

図15-3　緊急速報メッセージの例

手短な内容のイベント通知が好ましい状況はたくさんあります。二つの例を詳しく見てみましょう。セキュリティと並行性です。

セキュリティ

詳細な情報を明示的に問い合わせることを受信側に強制することで、保護すべき情報がメッセージ通信基盤上に漏洩することを防ぎます。そして、保護すべき情報へのアクセスには、追加の権限チェックを行います。

並行性

イベント駆動型の連係は非同期です。そのため、メッセージが届いた時には、その内容は最新ではなくなっている可能性があります。競合状態を厳密に扱う必要がある場合、明示的に問い合わせることで、最新の状態が手に入ります。

さらに言えば、複数のコンシューマーが競合している場合、イベントを一つのコンシューマーだけが処理するように、問い合わせに応答する側で、悲観的ロックを組み込むことが可能です。この方法を使えば、同じメッセージを複数のコンシューマーが処理することがないように、プロデューサー側が制御できます。

15.2.3.2　イベントによる状態転送

イベントの二つ目のカテゴリーは、イベントによる状態転送（Event-Carried State Transfer）メッセージです。このカテゴリーのメッセージは、プロデューサー内部の状態変更を購読側に通知します。イベント通知メッセージとは対照的に、このメッセージは状態の変更を反映したすべてのデータを含みます。

状態転送メッセージには二つの形式があります。一つは、次のような形式の、変更後の状態の完全なスナップショットです。

```
{
  "type": "customer-updated",
  "event-id": "6b7ce6c6-8587-4e4f-924a-cec028000ce6",
  "customer-id": "01b18d56-b79a-4873-ac99-3d9f767dbe61",
  "timestamp": 1615728520,
  "payload": {
    "first-name": "Carolyn",
    "last-name": "Hayes",
    "phone": "555-1022",
    "status": "follow-up-set",
    "follow-up-date": "2021/05/08",
    "birthday": "1982/04/05",
    "version": 7
  }
}
```

この例は、更新された顧客データの完全なスナップショットです。もし大きなデータ構造を操作している場合、以下のように、実際に更新されたフィールドだけをメッセージに含めるほうが適切かもしれません。

```
{
  "type": "customer-updated",
```

```
    "event-id": "6b7ce6c6-8587-4e4f-924a-cec028000ce6",
    "customer-id": "01b18d56-b79a-4873-ac99-3d9f767dbe61",
    "timestamp": 1615728520,
    "payload": {
      "status": "follow-up-set",
      "follow-up-date": "2021/05/10",
      "version": 8
    }
  }
```

状態転送メッセージの内容が、完全なスナップショットでも、更新されたフィールドだけでも、一連のメッセージを使って、対象となっているエンティティの状態をローカルにキャッシュすることが可能です。考え方としては、状態転送メッセージは、データを非同期に複製する仕組みです。この方法によって耐障害性が向上します。つまり、プロデューサーが止まっていても、コンシューマーは、ローカルに複製したデータを使って処理を継続できます。また、この方法は、複数の情報源からのデータを処理する必要があるコンポーネントの性能を向上します。データが必要になるたびに情報源に問い合わせるのではなく、すべてのデータをローカルにキャッシュしておけます（図15‐4）。

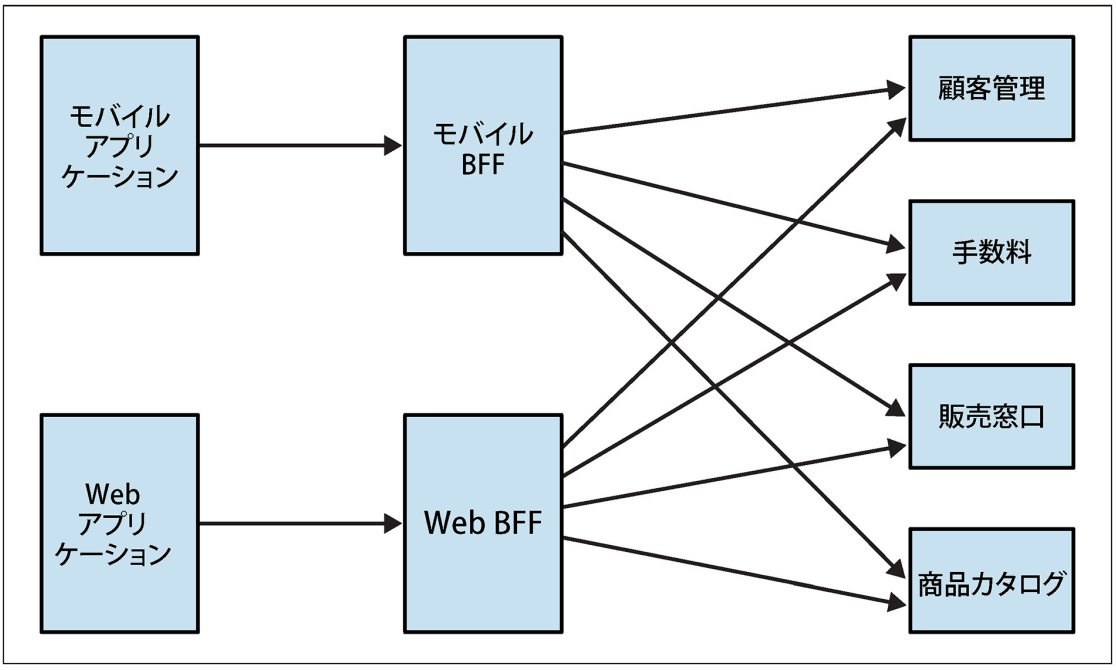

図15-4　データをローカルにキャッシュする例：バックエンドフォーフロントエンド（BFF : Backend For Frontend）

15.2.3.3　業務イベント

イベントメッセージの三つめのカテゴリーは、第6章で説明した、業務イベントです。見方によっては、業務イベントはイベント通知とイベントによる状態転送の中間の存在です。つまり、業務イベントは、業務領域で発生した重要な出来事を表現しつつ、その出来事を説明するすべてのデータを含みます。類似点はあるものの、三つのカテゴリーは概念として異なります。

15.2.3.4　業務イベントとイベント通知

業務イベントとイベント通知は、どちらもメッセージの発信元の業務領域で起きた変化を表現します。しかし、業務イベントは二つの点でイベント通知と異なります。

第一に、業務イベントはそのイベントを表現するすべての情報を含みます。受け取る側は、詳細情報を受け取るために、何か追加のアクションを起こす必要はありません。

第二に、モデルの意図が異なります。イベント通知の目的は、他のコンポーネントとの連係の複雑さの緩和です。それに対し、業務イベントの目的は、業務領域の活動をモデル化し説明することです。業務イベントは、他のコンポーネントがそのイベントに興味を持たなくても、役に立つことがあります。特にイベント履歴方式を取り入れたシステムでは、業務イベントは、業務領域で発生する状態の変化をモデル化することに役立ちます。しかし、他のコンポーネントがあらゆる業務イベントに興味を持つような設計は好ましくないでしょう。その理由はこの章の後半で詳しく検討します。

15.2.3.5　業務イベントとイベントによる状態転送

業務イベントに含まれるデータは、イベントによる状態転送のデータ構造とは異なります。

イベントによる状態転送では、発信側の状態を受信側でローカルにキャッシュするために十分な情報を提供します。それに対し、業務イベントはそのような詳細を外部に公開しません。購読していない他の業務イベントが、同じフィールドを変更している可能性があるため、購読している業務イベントだけでは、集約の状態をローカルにキャッシュできません。

さらに言えば、通知イベントの場合と同様に、業務イベントとイベントによる状態転送は、モデルの意図が異なります。業務イベントが表現するのは集約

の状態ではありません。業務イベントは集約のライフサイクルの途中で発生した業務的な出来事を表現します。

15.2.3.6　イベントのカテゴリー：具体例

イベントの三つのカテゴリーの違いを具体例で見てみましょう。結婚というイベントを三つの方法で表現してみます。

```
（イベント通知）
eventNotification = {
  "type": "婚姻届済",
  "person-id": "01b9a761",
  "payload": {
    "person-id": "126a7b61",
    "details": "/01b9a761/marriage-data"
  }
};

（イベントによる状態転送）
ecst = {
  "type": "個人情報変更済",
  "person-id": "01b9a761",
  "payload": {
    "new-last-name": "Williams"
  }
};

（業務イベント）
domainEvent = {
  "type": "結婚",
  "person-id": "01b9a761",
  "payload": {
    "person-id": "126a7b61",
    "assumed-partner-last-name": true
  }
};
```

`婚姻届済`はイベント通知メッセージです。メッセージの内容は、このIDの人が婚姻届けを提出した、という事実だけです。受信側が詳細な情報を知りたい時は、`details`フィールドに記載されたリンクを参照します。

`個人情報変更済`は、イベントによる状態転送です。このメッセージは、個人情報の変更、具体的には姓の変更を表現します。変更された理由は不明です。結婚したからかもしれませんし、離婚したからかもしれません。

最後の`結婚`は業務イベントです。現実世界で起きた出来事の要点を表現しています。個人を特定するIDと結婚相手の姓を名乗るかどうか（`assumed-partner-last-name`）を表現します。

15.3　イベント駆動型の連係を設計する

第3章で検討したように、ソフトウェア設計の主要な課題は境界です。境界は、内部に属するものとそうでないものを定義します。そして、もっとも重要な点として、境界を越えるものは何か、を明らかにします。つまり、コンポーネントどうしの連係方法を定義します。イベント駆動型のシステムでは、第一級の設計要素はイベントです。イベントの設計が、コンポーネントの連係方法とコンポーネント間の境界を明確にします。イベントメッセージの三つのカテゴリーのどれを選択するかで、分散システムが疎結合になるか、あるいは密結合になるかが決まります。

この節では、イベント種類をどう選択するかの経験則を学びます。まず、イベントの不適切な使い方によって、コンポーネントどうしが強く結合し、分散した大きな泥団子になってしまう例を見てみましょう。

15.3.1　分散した大きな泥団子

図15‐5のシステムを見てください。

このシステムは四つの区切られた文脈が関係しています。 顧客管理の業務ロジックの実装方法はイベント履歴式ドメインモデルです。顧客管理と販売促進の連係が必要になったので、二つのチームは、イベント履歴式ドメインモデルの柔軟性を活用することにしました。販売促進が顧客管理の業務イベントを購読し、その業務イベントを使って、販売促進が必要とするモデルを投影することにしました。

その後、別の区切られた文脈として、広告最適化を導入しました。広告最適化も顧客管理の文脈で発生する情報を処理することが必要です。この時も、広告最適化が顧客管理のすべての業務イベントを購読し、広告最適化が必要とするモデルを投影することにしました。

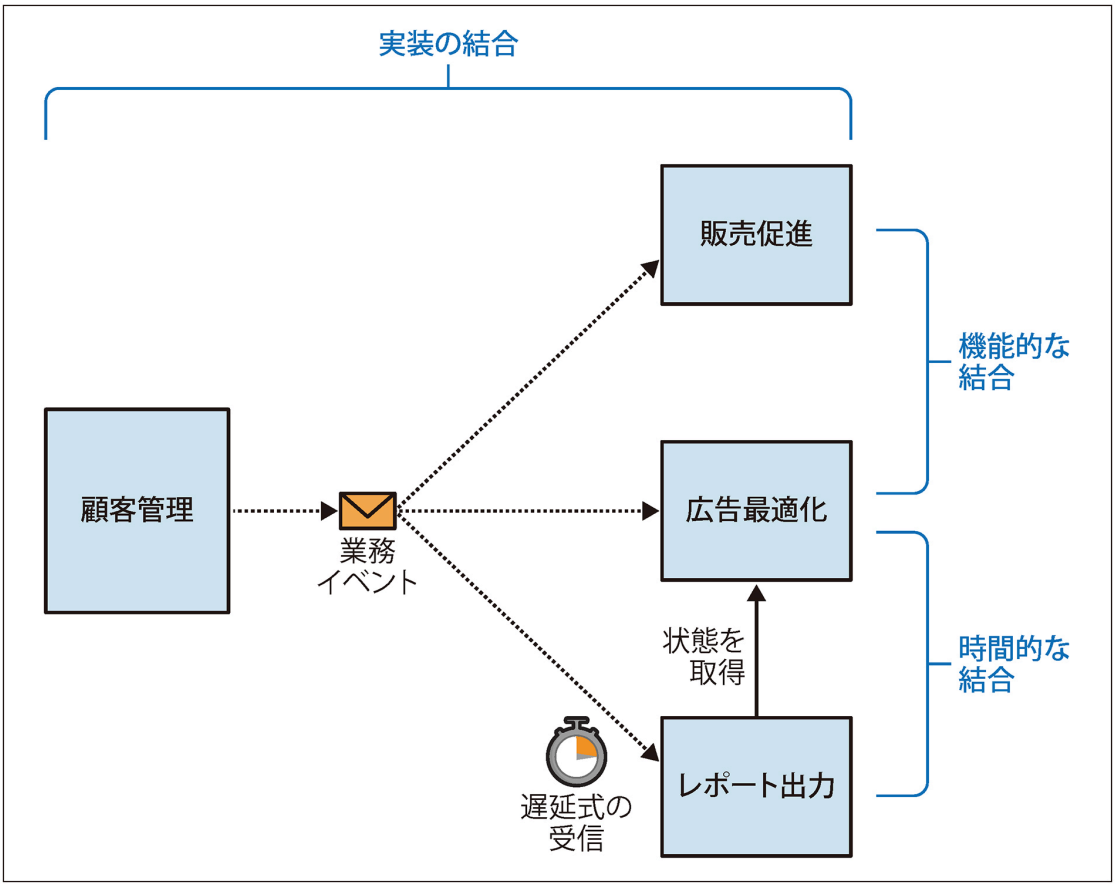

図15-5　密結合した分散システム

とても興味深いことに、販売促進と広告最適化が必要とする顧客情報は同じ形式でした。そのため、この二つの区切られた文脈は、顧客管理の業務イベントから同じモデル、つまり顧客の最新状態をべたがきしたスナップショットを投影していました。

もう一つの区切られた文脈であるレポート出力は、顧客管理が発行する一部の業務イベントを購読し、イベント通知メッセージとして利用していました。このイベントを検知したタイミングで、広告最適化から計算結果を取得する仕組みです。しかし、広告最適化とレポート出力は同じイベントを使って処理を開始していました。そのため、レポート出力のモデルが最新になることを保証するために、レポート出力処理の開始を遅延するようにしました。顧客管理の業務イベントを受け取ってから5分後に処理を開始する仕組みです。

これは、かなりひどい設計です。このシステムでどんな結合が起きているか分析してみましょう。

15.3.2　時間的な結合

広告最適化とレポート出力の二つの区切られた文脈は、時間的に結合しています。つまり、実行の順番に強く依存しています。広告最適化の計算処理は、レポート出力の実行開始よりも前に完了していなければなりません。順番が逆になると、最新の状態を反映していないレポートが出力されます。

確実に正しい順番で処理するために、開発チームはレポート出力に遅延の仕組みを導入しました。5分間の遅延があれば、広告最適化に必要な計算処理は完了する、というもくろみです。明らかに、このやり方で順番の不正を防ぐことはできません。次のような状況が考えられます。

- 広告最適化の負荷が高くなれば、計算処理を5分で完了できない可能性がある
- ネットワーク障害で、広告最適化サービスへの業務イベントの配信が遅れる可能性がある
- 広告最適化で何らかの障害が発生し、受信メッセージの処理が止まっている可能性がある

15.3.3　機能的な結合

販売促進と広告最適化の二つの区切られた文脈は、どちらも顧客管理の業務イベントを購読しています。その結果、それぞれが顧客データの投影処理を実装しています。別の言い方をすると、到着した業務イベントから顧客の状態を投影する業務ロジックが二つの区切られた文脈で重複しています。重複して実装された業務ロジックは同じ理由で変更されます。二つの文脈は同じ形式で顧客情報を表示することが必要です。したがって、もしどちらか片方の文脈で投影方法を変更したら、もう一つの文脈でも同じ変更が必要です。

これが機能的な結合の具体例です。つまり、複数のコンポーネントが同じ業務機能を実装しています。そして、機能の変更があれば、両方のコンポーネントを同時に変更することが必要です。

15.3.4　実装の結合

実装の結合は、かなりとらえにくい関係です。販売促進と広告最適化の二つの区切られた文脈は、顧客管理のイベント履歴式ドメインモデルが生成するすべての業務イベントを購読します。その結果、顧客管理側で業務イベントの追加やデータ構造の変更があると、購読側の二つの区切られた文脈もそれに対応した変更が必要になってしまいます。もし対応しなければ、データの不整合が

発生します。たとえば、業務イベントのデータ構造が変更された場合、購読側の状態を投影するロジックでエラーが発生します。あるいは、新しい業務イベントを顧客管理モデルへ追加した場合、受信側の投影モデルに潜在的な影響を与えます。追加された業務イベントを無視することになり、正しくない状態が投影されることになるでしょう。

15.3.5　イベント駆動型連係のリファクタリング

ここまで見てきたように、システムをイベントだらけにすると、密結合になり、弾力性が失われます。非現実的な例に見えるかもしれません。しかし、残念ながら、この例は実際にあった話にもとづいています。では、イベントをどのように使えば、このひどい設計を劇的に改善できるかを見てみましょう。

顧客管理のモデルを構成するすべての業務イベントを公開してしまうと、イベントの購読側は顧客管理の実装の詳細と密に結合します。このような実装の結合は、公開する業務イベントをごく一部に制限するか、業務イベントとは異なる種類のイベントを公開することで対処します。

販売促進と広告最適化は、同じイベントを購読し、同じ業務機能を実装しているために、双方向で機能的に結合しています。

実装の結合と機能的な結合のどちらの場合も、投影ロジックを発信側、つまり顧客管理側にカプセル化することで改善できます。顧客管理は、実装の詳細を公開するかわりに、コンシューマー駆動契約（Consumer-Driven Contract）を取り入れます。つまり、受信側が必要とするモデルを顧客管理が投影し、その投影結果を公開された言葉に含めることで、他の区切られた文脈との連係に特化したモデルを提供します。連係用のモデルは、顧客管理内部の実装モデルから切り離されます。その結果、受信側は必要なすべてのデータを入手し、顧客管理の実装モデルを意識しなくなります。

広告最適化とレポート出力の二つの区切られた文脈の間にある時間的な結合を取り除くために、広告最適化がイベント通知メッセージを発行し、レポート出力はその通知を受信した時に必要なデータを取りに行くという方法があります。以上を適用して改善したシステムは**図15‐6**のようになります。

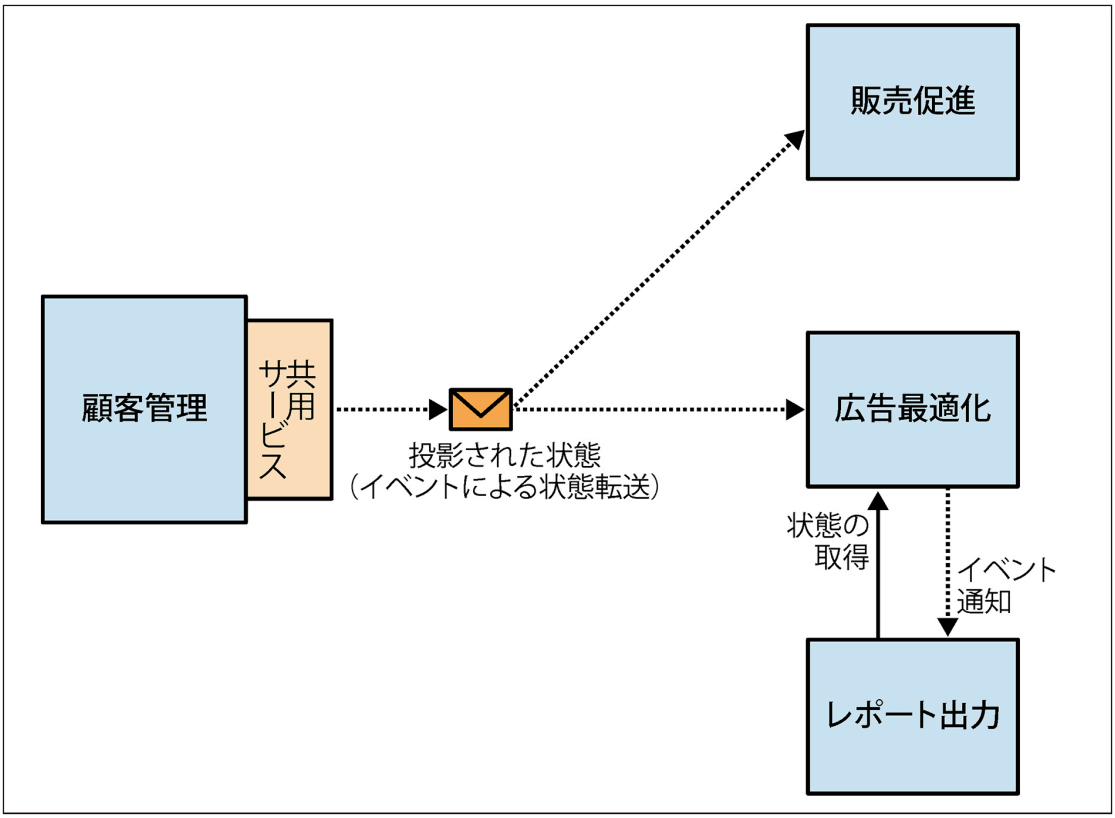

図15-6　改善されたシステム

15.3.6　イベント駆動型で設計する経験則

目の前の課題に対して適切なイベントのカテゴリーを選択することで、桁違いによい設計になります。疎結合になり、柔軟になり、耐障害性が向上します。このような改善の背後にある設計の経験則を言語化してみましょう。

15.3.6.1　最悪のケースを想定する

アンドルー・グローブが書いたようにパラノイア（病的な心配性）だけが生き残ります＊4。イベント駆動型システムを設計する時は、この考え方を指針にしてください。

＊4　A. S. Grove著『Only the Paranoid Survive』（HarperCollins Business、1998年）（邦訳『パラノイアだけが生き残る』日経BP社、2017年）

- ネットワークは必ず遅延する
- サーバーはもっとも都合が悪い時に故障する
- イベントは順番通りには届かない
- イベントは重複する

そして忘れてはいけません。こういう障害が起きるのは、決まって休日や祝祭日です。

イベント駆動型アーキテクチャの「駆動」が意味するのは、システム全体が、メッセージが確実に届くことに依存しているということです。ですから「うまくいくだろう」という期待は捨ててください。何があっても、イベントが確実に届く仕組みを工夫してください。つまり、次のような方法です。

- メッセージを確実に送出するために、送信箱（Outbox）を使う
- メッセージを発行する場合、メッセージを購読する側が重複したメッセージを取り除き、ばらばらに届くメッセージを正しい順番に並べ直せることを保証する
- 複数の区切られた文脈にまたがる、補償アクションが必要な業務プロセスでは、サーガまたはプロセスマネージャーを活用する

15.3.6.2　公開イベントと内部イベントを使い分ける

業務イベントを発行する時は、実装の詳細が外部に漏れ出さないようにしてください。特にイベント履歴式の集約では注意が必要です。そして、イベントを区切られた文脈の公開インターフェースの不可欠な要素として扱ってください。つまり、共用サービスを実装する場合、イベントが区切られた文脈の公開された言葉を反映するようにしてください。イベントベースのモデルを変換するさまざまなやり方は、**第9章**で検討しました。

区切られた文脈の公開インターフェースを設計する時は、イベントのカテゴリーの違いをうまく使ってください。イベントによる状態転送メッセージは、実装モデルを圧縮し、受信側が必要とする最小限の情報を伝達します。

イベント通知メッセージを使えば、公開インターフェースをさらに小さくできます。

最後に、業務イベントを他の区切られた文脈との通信にそのまま使うのは、控えめにしましょう。公開する業務イベントを限定することを検討してください。

15.3.6.3　一貫性の要求レベルを検討する

イベント駆動型の通信を設計する時に、どのカテゴリーのイベントを使えばよいかを判断するための経験則が、もう一つあります。区切られた文脈の間で、どのような一貫性が必要か詳しく調べてください。次のやり方です。

- それぞれの区切られた文脈で、最終的に一貫性のあるデータに落ち着けばよいのであれば（結果整合性）、イベントによる状態転送メッセージを使う
- 受信側で、発信側の最新の書き込みを読み取る必要があれば、イベント通知メッセージを発行し、発信側の最新状態を入手するための問い合わせを実行する

15.4　まとめ

この章では、区切られた文脈の公開インターフェースを設計する時の本質的な側面として、イベント駆動型アーキテクチャを紹介しました。区切られた文脈の境界を越えて通信する以下の3種類のイベントを学びました。

イベント通知

何か重要な出来事が発生したことを伝えます。しかし、詳細な情報は、受信側が発信元に問い合わせることが必要です。

イベントによる状態転送

メッセージを使ってデータを非同期に複製する仕組みです。それぞれのイベントは、ある状態のスナップショットを伝えます。受信側は、ローカルにキャッシュした発信元のデータを最新に維持するために、このスナップショットを利用できます。

業務イベント

発信側の業務領域で発生した出来事（イベント）を表現したメッセージです。

どのカテゴリーのイベントを使うかの選択をまちがえると、イベント駆動型のシステムは失敗するでしょう。気がつかないうちに、全体が大きな泥団子になってしまいます。連係するための適切なイベント種類を選ぶには、区切られた文脈の間で、どの程度のデータの一貫性が必要かを詳しく調べます。文脈内部の実装の詳細を他の文脈に露出させないように気をつけてください。そし

て、障害の発生や一部のシステムの停止があっても、確実にメッセージを届けられる仕組みを用意してください。

15.5　演習問題

1. 次の説明で正しいのはどれですか？
   A. イベント駆動型アーキテクチャでは、コンポーネントの境界を越えて情報をやりとりするためのイベントを定義する
   B. イベントソーシングでは、区切られた文脈の内部でしか使わないイベントを定義する
   C. イベント駆動型アーキテクチャとイベントソーシングは、同じ設計方法の別名である
   D. AとBが正しい
2. 状態の変更内容を伝えるために、もっとも適切なイベントのカテゴリーはどれですか？
   A. イベント通知
   B. イベントによる状態転送
   C. 業務イベント
   D. 状態の変更内容を伝える手段として、どのイベントカテゴリーも同じように適切である
3. 区切られた文脈をつなぐための公開イベントを明示的に設計するのは、どの連係方法ですか？
   A. 共用サービス
   B. モデル変換装置
   C. モデルの共有
   D. 従属
4. サービス1（S1）とサービス2（S2）を非同期でつないでいます。S1はデータを外部に伝える必要があり、S2はS1の最新の状態を知る必要があります。この場合、どのイベントカテゴリーを使うのが適切ですか？
   A. S1がイベントによる状態転送メッセージを発行する
   B. S1がイベント通知メッセージを発行し、メッセージを受け取ったS2がS1に最新状態を問い合わせる
   C. S1が業務イベントを発行する
   D. AとBが正しい